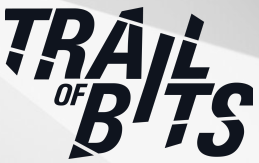

The Quantum Resistant Ledger go-qrllib Library

Security Assessment

July 8, 2026

Prepared for:
Die QRL Stiftung
The Quantum Resistant Ledger

Prepared by: **Filipe Casal, Marc Ilunga, and Robin Sandblom**

Table of Contents

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Jim Miller, Engineering Director, Application Security and Cryptography
james.miller@trailofbits.com

The following consultants were associated with this project:

Filipe Casal, Consultant
filipe.casal@trailofbits.com

Marc Ilunga, Consultant
marc.ilunga@trailofbits.com

Robin Sandblom, Consultant
robin.sandblom@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
April 6, 2026	Pre-project kickoff call
April 23, 2026	Delivery of report draft and report readout meeting
May 13, 2026	Delivery of final comprehensive report
July 8, 2026	Delivery of final comprehensive report with fix review

Executive Summary

Engagement Overview

The Quantum Resistant Ledger (QRL) engaged Trail of Bits to review the security of the `go-qrllib` library, a post-quantum digital signature and wallet library for the QRL blockchain.

A team of three consultants conducted the review from April 13 to April 22, 2026, for a total of three engineer-weeks of effort. Our testing efforts focused on the correctness of the ML-DSA and XMSS implementations relative to their specifications, the integrity of the exported API boundary, and the safety of the wallet and legacy wallet state-management paths. With full access to the source code and documentation, we conducted static and dynamic testing of the codebase using both automated and manual processes. The integration and usage of the `go-qrllib` library in other codebases was out of scope for this review.

Observations and Impact

Overall, we found the codebase to be well-organized and modular. The implemented cryptographic primitives strongly follow the respective specifications. The codebase has strong testing coverage. The wallet is implemented defensively for usage in the QRL ecosystem.

The most noteworthy issue (TOB-QRLLIB-13) is an XMSS tree-initialization flaw that leads to degenerate keys and signatures. We identified API misuse risks, including misbehavior on otherwise valid RFC 8391 parameter sets (TOB-QRLLIB-1, TOB-QRLLIB-2). In the legacy wallet, the `XMSSWallet`, the `sign` wrapper hides critical warnings for sensitive operations, such as XMSS signing (TOB-QRLLIB-8).

On the ML-DSA side, the public `Open` and `Verify` functions panic with a nil public key (TOB-QRLLIB-11), and we identified inconsistent zeroization of ML-DSA secret memory (TOB-QRLLIB-10), a misleading `Seal` name for a signature-only operation (TOB-QRLLIB-13), and a C-to-Go operator-precedence porting hazard (TOB-QRLLIB-10).

Additional informational issues include the use of a non-FIPS-compliant hash-function parameter with reduced quantum security (TOB-QRLLIB-7), disabling of hedged signing in the ML-DSA path (TOB-QRLLIB-6), and acceptance of SPHINCS+ as a valid signature scheme in the wallet descriptor (TOB-QRLLIB-4).

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that The Quantum Resistant Ledger take the following steps:

- **Remediate the findings disclosed in this report.** The identified issues primarily affect the robustness of the exported cryptographic API rather than the correctness of the underlying primitives, and should be addressed through direct fixes or a focused refactor of the library's validation boundary.
- **Establish and enforce explicit precondition validation at the exported API surface.** Every public entry point in the ML-DSA, XMSS, wallet, and legacy wallet packages should validate nil inputs, parameter-set identifiers, hash function identifiers, and typed bounds before operating on cryptographic state, so that invalid inputs produce deterministic errors rather than panics or silently degraded output. Additionally, extensively document the public API.
- **Extend the testing infrastructure.** Integrate the structured and metamorphic fuzzers, and the ACVP and Wycheproof test vector suites exercised into the testing infrastructure of the codebase, so that regressions in the primitives or their boundary are caught before release.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	1
Medium	0
Low	4
Informational	10
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Auditing and Logging	1
Configuration	3
Cryptography	4
Data Exposure	1
Data Validation	3
Denial of Service	1
Error Reporting	2

Project Goals

The engagement was scoped to provide a security assessment of the Quantum Resistant Ledger `go-qrllib` library. Specifically, we sought to answer the following non-exhaustive list of questions:

- Are XMSS and ML-DSA securely implemented in accordance with their specifications?
- Does the ML-DSA-87 implementation follow the FIPS 204 specification?
- Are XMSS index state management and persistence handled correctly?
- Are there any side-channel vulnerabilities in the cryptographic implementations?
- Are mathematical primitives (polynomials, lattices, hash functions) correctly implemented and used?
- Is the library's public API robust against malicious or malformed inputs?

Project Targets

The engagement involved reviewing and testing the following target.

go-qrllib

Repository	https://github.com/theQRL/go-qrllib
Version	2289c72110b809724be3de46f324e9d99ea2bd47
Type	Go

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **Manual source review**
  - Spec-to-implementation comparison against FIPS 204 for ML-DSA-87
  - Spec-to-implementation comparison against RFC 8391 and NIST SP 800-208 for XMSS
  - Cross-implementation comparison against the respective reference codebases and other public implementations
  - Targeted API-contract review covering parsing, error handling, `nil` handling, naming, and validation boundary inconsistencies
- **Fuzzing**
  - Fuzz corpora extended for both ML-DSA-87 and XMSS with:
    - Structured round-trip fuzzers
    - Mutation-based fuzzers
    - Metamorphic fuzzers inspired by the techniques, *Finding Bugs and Features Using Cryptographically-Informed Functional Testing*: hand-written metamorphic test suites asserting cryptographic invariants of ML-DSA-87
- **Mutation testing**
  - Applied to XMSS and ML-DSA to measure test-suite coverage of behavior that static coverage misses
- **External test-vector integration**
  - NIST ACVP ML-DSA vectors
  - Wycheproof ML-DSA verification vectors
  - Both integrated into the Go test harness

- **cgo-backed C-reference equivalence**
    - Equivalence testing and fuzzing against the C reference implementation for low-level arithmetic and helper functions, including `MontgomeryReduce` and `UseHint`
- **Cross-architecture assembly inspection**
    - Generated assembly for critical helpers inspected for:
        - Secret-dependent branching
        - Division/remainder instructions
        - Constant-time-preserving codegen shape
- **C-to-Go operator-precedence static analysis**
    - A bespoke tool built for this audit to detect operator-precedence porting hazards in expressions translated from the C reference

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Test Harness Configuration

We used the following tools in the automated testing phase of this project. The setup and usage options for all tools are described in appendix D.

Tool	Description
CodeQL	A code analysis engine developed by GitHub to automate security checks
Semgrep	An open-source static analysis tool for finding bugs and enforcing code standards when editing or committing code and during build time
zizmor	A static analysis tool for GitHub Actions
Native Go fuzzing	Quickly set up a fuzzing campaign in Go
`golangci-lint`	A meta-linter for Go that runs multiple linters and static analysis tools on the codebase and aggregates the results
Go cover	A tool that generates test coverage reports for Go codebases
Gremlins	A mutation testing tool for Go

Areas of Focus

Our automated testing and verification work focused on the following:

- General code quality issues and unidiomatic code patterns
- Data validation issues with fuzzing
- Security of CI and GitHub actions
- Dependencies that are outdated or vulnerable to known attacks

- Gaps in test coverage

Test Results

The results of this focused testing are detailed below.

CodeQL

We did not identify any issue using the Go CodeQL rules.

Semgrep

We did not identify any issues using the Go Semgrep rules.

zizmor

Running zizmor on the codebase identified TOB-QRLLIB-15. We recommend including zizmor in the CI/CD pipeline of the codebase.

golangci-lint

We did not identify any issues using golangci-lint.

go-mod-outdated

We ran `go-mod-outdated` to identify outdated dependencies. The tool flagged that golang.org/x/crypto is currently on version `0.48.0`. There are no known security advisories for this version; the team should consider updating if appropriate.

Go cover

The codebase has generally strong test coverage across the exercised packages, with especially high coverage in the core cryptographic implementations. A small set of packages has lower test coverage: common, crypto/errors, legacywallet/common, and qrl.

Package	Coverage
common	—
crypto/dilithium	90.4%
crypto/errors	—
crypto/internal/lattice	100.0%
crypto/ml_dsa_87	91.1%
crypto/sphincsplus_256s	95.2%
crypto/sphincsplus_256s/params	90.0%

crypto/xmss	99.0%
legacywallet	100.0%
legacywallet/common	—
legacywallet/xmss	98.4%
misc	97.3%
qrl	—
wallet	85.0%
wallet/common	94.3%
wallet/common/descriptor	100.0%
wallet/common/wallettype	100.0%
wallet/misc	97.9%
wallet/ml_dsa_87	94.7%
wallet/sphincsplus_256s	94.9%

Aggregated testing coverage

Gremlins

Gremlins identified non-RFC conformance issues reported in TOB-QRLLIB-1.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	XMSS implementation is not fully compliant with RFC 8391	Data Validation	**Low**
2	XMSS InitializeTree accepts invalid typed Height values above MaxHeight	Data Validation	**Low**
3	Modern wallet signatures do not bind descriptor metadata	Data Validation	**Informational**
4	Wallet allows SPHINCS+ as valid signature scheme	Configuration	**Informational**
5	prf function hardcodes input length without validating the actual input length	Configuration	**Informational**
6	ML-DSA implementation uses deterministic signing	Configuration	**Informational**
7	SHAKE_128 is a non-compliant FIPS parameter with reduced quantum security	Cryptography	**Informational**
8	Exported XMSSWallet API drops security-critical documentation	Cryptography	**Informational**
9	C-to-Go translation can silently change semantics due to operator precedence differences	Cryptography	**Informational**
10	ML-DSA secret-memory cleanup is inconsistent across functions	Data Exposure	**Informational**
11	ML-DSA Open and Verify panic on a nil public key	Denial of Service	**Low**
12	ML-DSA Seal naming implies confidentiality for a signature-only operation	Error Reporting	**Informational**

13	Invalid XMSS hash function values produce degenerate interchangeable keys and signatures	Cryptography	**High**
14	ML-DSA Open API collapses distinct failure modes into a nil result	Error Reporting	**Informational**
15	Unpinned external GitHub CI/CD action versions	Auditing and Logging	**Low**

Detailed Findings

1. XMSS implementation is not fully compliant with RFC 8391

Severity: **Low**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-QRLLIB-1
Target: `crypto/xmss/xmss_fast.go`, `crypto/xmss/params.go`, `crypto/xmss/const.go`, `crypto/xmss/doc.go`	

Description

The `crypto/xmss` package advertises RFC 8391 XMSS support, but its exported parametric key-generation API silently produces malformed key pairs whose secret keys cannot sign under the corresponding public keys. Despite the parametric API, `XMSSFastGenKeyPair` hardcodes `rnd = 96` and `pks = uint32(32)` as shown in figure 1.1, which only match the fixed `n = 32` layout.

```
func XMSSFastGenKeyPair(hashFunction HashFunction, xmssParams *XMSSParams,
	pk, sk []uint8, bdsState *BDSState, seed []uint8) error {

	if xmssParams.h&1 == 1 {
		//coverage:ignore
		//rationale: InitializeTree validates height with BDS check which
ensures even heights before calling this
		return cryptoerrors.ErrInvalidHeight
	}
	n := xmssParams.n
[...]
	misc.SHAKE256(randombits, seed)
	//shake256(randombits, 3 * n, seed, 48);  // FIXME: seed size has been
hardcoded to 48

	rnd := 96
	pks := uint32(32)
	copy(sk[4:], randombits[:rnd])
	copy(pk[n:], sk[4+2*n:4+2*n+pks])
[...]
}
```

Figure 1.1: XMSS key generation with hardcoded constants (go-qrllib/crypto/xmss/xmss_fast.go#L8–L40)

A comment in the codebase indicates that the team is aware of the assumptions but has not yet addressed the issue. Other instances of the same assumptions are in `const.go`.

The issue has no impact with respect to how XMSS is used in the wallet. However, the hardcoded logic induces several failure modes for third-party users, including:

- Runtime panics when encountering invalid parameter configurations, especially when n is less than 32, which triggers an out-of-bounds memory read; and
- Failure to verify signatures for n = 64. `Verify` and `VerifyWithCustomWOTSParamW` also assume `n = 32`, which breaks interoperability with other implementations.

Exploit Scenario

A downstream protocol uses `go-qrllib` with parametric XMSS addresses. An attacker convinces Alice to migrate to `n = 64` for stronger post-quantum security. Alice transfers all her funds to the new address, and they are permanently stranded there. Figure 1.2 shows a key generation for the parameter set (`n = 64, h = 10, w = 16, k = 2, SHAKE_256`) resulting in a `sk` buffer with an all-zero `pub_seed` field and a root truncated to 32 of its 64 bytes.

```
func TestXMSSFastGenKeyPairN64ParametricBuffersProduceMalformedOutput(t *testing.T)
{
	const n = uint32(64)
	params := NewXMSSParams(n, 10, 16, 2)
	bdsState := NewBDSState(10, n, 2)
	sk := make([]uint8, 4+4*n)
	pk := make([]uint8, 2*n)
	seed := bytes.Repeat([]byte{0x24}, int(3*n))
	if err := XMSSFastGenKeyPair(SHAKE_256, params, pk, sk, bdsState, seed); err
!= nil {
		t.Fatalf("XMSSFastGenKeyPair returned unexpected error: %v", err)
	}
	if !bytes.Equal(sk[4+2*n:4+3*n], make([]byte, n)) {
		t.Fatalf("pub_seed region in secret key should remain zero for n=64,
got %x", sk[4+2*n:4+3*n])
	}
	if !bytes.Equal(sk[4+3*n+32:4+4*n], make([]byte, 32)) {
		t.Fatalf("root tail in secret key should remain zero for n=64, got %x",
sk[4+3*n+32:4+4*n])
	}
	if !bytes.Equal(pk[n+32:2*n], make([]byte, 32)) {
		t.Fatalf("public key tail should remain zero for n=64, got %x",
pk[n+32:2*n])
	}
}
```

Figure 1.2: A successful key generation test for n = 64, h = 10, w = 16, and k = 2

Recommendations

Short term, reject unsupported parameter tuples at the exported boundary and document the supported parameters consistently.

Long term, ensure the implementation matches the advertised specifications. Use known answer tests and negative tests to ensure the library works as expected. Consider fuzzing the library and inspecting any resulting failures.

2. XMSS InitializeTree accepts invalid typed Height values above MaxHeight

Severity: **Low**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-QRLLIB-2
Target: `crypto/xmss/height.go`, `crypto/xmss/xmss.go`, `crypto/xmss/xmss_fast.go`	

Description

The XMSS package declares a typed `Height` value with a validity contract of even heights between 2 and `MaxHeight = 30`, but its public constructor `InitializeTree` never enforces that contract. A caller who builds a `Height` with a direct cast (e.g., `xmss.Height(32)`) can construct a seemingly valid XMSS whose Merkle root is all zeros and which fails only later at signing time.

Figure 2.1 shows that `InitializeTree` accepts a typed value directly, does not call `h.IsValid()` and only the BDS relation `k < h` and `(h-k) % 2 == 0`.

```
func InitializeTree(h Height, hashFunction HashFunction, seed []uint8) (*XMSS,
error) {
	height := uint32(h)
[....]
	if k >= height || (height-k)%2 == 1 {
		return nil, cryptoerrors.ErrInvalidBDSParams
	}

	xmssParams := NewXMSSParams(n, height, w, k)
	bdsState := NewBDSState(height, n, k)

	if err := XMSSFastGenKeyPair(hashFunction, xmssParams, pk, sk, bdsState,
seed); err != nil {
		//coverage:ignore
		//rationale: XMSSFastGenKeyPair only fails for odd heights, but BDS
check above ensures heights are even
		return nil, cryptoerrors.ErrKeyGeneration
	}
```

Figure 2.1: `InitializeTree` does not fully validate the height. (go-qrllib/crypto/xmss/xmss.go#L24–L44)

Out-of-range even heights reach `treeHashSetup`, where `lastNode := index + (1 << h)` wraps on `uint32` for `h >= 32`; the Merkle-build loop guarded by `index < lastNode` then runs zero or a handful of iterations, and the root field is left in its zero-initialized state.

```
lastNode := index + (1 << h)

bound := h - k
stack := make([]uint8, (h+1)*n)
stackLevels := make([]uint32, h+1)
stackOffset := uint32(0)
nodeH := uint32(0)

for i := uint32(0); i < bound; i++ {
       bdsState.treeHash[i].h = i
       bdsState.treeHash[i].completed = 1
       bdsState.treeHash[i].stackUsage = 0
}

i := uint32(0)
for ; index < lastNode; index++ {
```

Figure 2.2: go-qrllib/crypto/xmss/xmss_fast.go#L130–L145

The package documentation is inconsistent: the README validates with `ToHeight` before `InitializeTree`, while the godoc shows the unvalidated literal form. External consumers of `crypto/xmss` are therefore the primary affected population; the legacy wallet path already validates height.

Exploit Scenario

A downstream service retrieves tree height from configuration or an API and casts it directly with `xmss.Height(cfg.Height)` before calling `InitializeTree`, per the godoc-style usage. An out-of-range value slips past construction, and the defect surfaces only at signing time. Figure 2.3 shows that certain even `Height` values above `MaxHeight` yield a zero-rooted XMSS.

```
func TestHeight_BypassAboveMaxHeight(t *testing.T) {
    seed := bytes.Repeat([]byte{0x42}, 48)
    for _, h := range []uint8{32, 34, 98} {
        t.Run(fmt.Sprintf("h=%d", h), func(t *testing.T) {
            tree, err := xmss.InitializeTree(xmss.Height(h), xmss.SHAKE_256, seed)
            if err != nil {
                t.Fatalf("InitializeTree h=%d unexpected err=%v", h, err)
            }
            if !bytes.Equal(tree.GetRoot(), make([]byte, 32)) {
                t.Fatalf("h=%d expected all-zero root, got %x", h, tree.GetRoot())
            }
        })
    }
}
```

Figure 2.3: Invalid height test case

Recommendations

Short term, call `h.IsValid()` at the top of `InitializeTree` and return `ErrInvalidHeight` before the BDS check, add a defense-in-depth guard in `XMSSFastGenKeyPair` or `treeHashSetup`, and update the godoc to use `ToHeight` rather than raw literals.

Long term, align the `InitializeTree` contract with the helper APIs and add regression tests that assert immediate `ErrInvalidHeight` for `Height(31)`, `Height(32)`, `Height(34)`, and `Height(98)` alongside a `Height(10)` round-trip control.

3. Modern wallet signatures do not bind descriptor metadata

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Data Validation	Finding ID: TOB-QRLLIB-3
Target: `crypto/xmss/height.go`, `crypto/xmss/xmss.go`, `crypto/xmss/xmss_fast.go`	

Description

The modern wallet API derives addresses by hashing the descriptor together with the public key, but wallet-level `Verify` only validates the descriptor's algorithm family and ignores the two metadata bytes. A single keypair therefore maps to many distinct addresses, while the same `(message, signature, pk)` tuple verifies under every same-family descriptor.

```
func UnsafeGetAddress(pk []byte, desc descriptor.Descriptor) [AddressSize]byte {
	sh := sha3.NewShake256()
	_, _ = sh.Write(desc.ToBytes())
	_, _ = sh.Write(pk)

	var addr [AddressSize]byte
	_, _ = sh.Read(addr[:]) // take the first N bytes
	return addr
}
```

Figure 3.1: Address derivation commits to the full three-byte descriptor. (go-qrllib/wallet/common/address.go#L23–L31)

As figure 3.1 shows, addresses are a function of a public key and the descriptor. Figure 3.2 shows that signature verification for ML-DSA involves validating the descriptor. The descriptor validator constrains only the type byte; the two metadata bytes are accepted unconditionally, as figure 3.3 shows.

```
func Verify(message, signature []uint8, pk *PK, desc
[descriptor.DescriptorSize]byte) (result bool) {
	_, err := NewMLDSA87DescriptorFromDescriptorBytes(desc)
[...]
	return ml_dsa_87.Verify(ctx, message, sig, &pk2)
}
```

Figure 3.2: Signature verification function for ML-DSA (go-qrllib/wallet/ml_dsa_87/wallet.go#L198–L216)

```
func (d Descriptor) IsValid() bool {
```

```
        switch wallettype.WalletType(d[0]) {
        case wallettype.SPHINCSPLUS_256S, wallettype.ML_DSA_87:
                return true
        default:
                return false
        }
}
```

Figure 3.3: Descriptor validation (go-qrllib/wallet/common/descriptor/descriptor.go#L49–L64)

The wallet's operations are not impacted. The library's own constructors emit fixed metadata. Furthermore, the QRL execution layer requires that the descriptor be included in the transaction hash that any sender signs, thereby mitigating signature reuse across addresses. However, this mitigation is implemented outside of the library and is not documented.

Recommendations

Short term, either reject non-zero metadata bytes for ML-DSA-87 and SPHINCS+256s descriptors, or bind those bytes into wallet-level `Verify` so a signature is accepted only under the exact descriptor-derived identity enrolled.

Long term, pick a single identity contract (public key alone vs. descriptor + public key) and enforce it uniformly across address derivation, extended-seed handling, wallet construction, and verification.

4. Wallet allows SPHINCS+ as valid signature scheme

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Configuration	Finding ID: TOB-QRLLIB-4
Target: `wallet/common/descriptor/descriptor.go`	

Description

The Descriptor allows for wallets to use SPHINCS+ as a signature scheme. The SPHINCS+ wallet type should no longer be supported.

```
func (d Descriptor) IsValid() bool {
	switch wallettype.WalletType(d[0]) {
	case wallettype.SPHINCSPLUS_256S, wallettype.ML_DSA_87:
		return true
	default:
		return false
	}
}
```

Figure 4.1: go-qrllib/wallet/common/descriptor/descriptor.go#L49–L56

Recommendations

Short term, remove the SPHINCS+ wallet type as a valid option.

Long term, add test coverage to ensure that only ML-DSA87 is a valid signature scheme for the wallet.

5. prf function hardcodes input length without validating the actual input length

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Configuration	Finding ID: TOB-QRLLIB-5
Target: `crypto/xmss/hash.go`	

Description

The `prf` function passes a hardcoded `inLen=32` to `coreHash` regardless of the actual length of the `in` slice. The `coreHash` function then iterates over `inLen`. If `len(in) < 32`, the access panics with a runtime index-out-of-range error.

```
func prf(hashFunction HashFunction, out, in, key []uint8, keyLen uint32) {
	coreHash(hashFunction, out, 3, key, keyLen, in, 32, keyLen)
}
```

Figure 5.1:go-qrllib/crypto/xmss/hash.go#L29–L31

```
func coreHash(hashFunction HashFunction, out []uint8, typeValue uint32, key []uint8,
keyLen uint32, in []uint8, inLen uint32, n uint32) {
	buf := make([]uint8, inLen+n+keyLen)
	misc.ToByteBigEndian(buf, typeValue, n) // RFC 8391 requires big-endian
encoding

	for i := uint32(0); i < keyLen; i++ {
		buf[i+n] = key[i]
	}

	for i := uint32(0); i < inLen; i++ {
		buf[keyLen+n+i] = in[i]
	}
```

Figure 5.2: go-qrllib/crypto/xmss/hash.go#L33–L43

All five current call sites already pass slices of 32-byte stack-allocated arrays (`idxBytes32[:]`, `bytes[:]`, `ctr[:]`, `byteAddr[:]`). However, if a future caller passes a shorter slice, the function would panic.

Recommendations

Short term, change the `in` parameter to a fixed-size array pointer: `func prf(hashFunction HashFunction, out []uint8, in *[32]uint8, key []uint8, keyLen uint32)`. This enforces the constraint during the compilation phase and makes the asymmetry with `key`/`keyLen` explicit.

6. ML-DSA implementation uses deterministic signing

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Configuration	Finding ID: TOB-QRLLIB-6
Target: `crypto/ml_dsa_87/ml_dsa_87.go`, `crypto/ml_dsa_87/sign.go`, `crypto/ml_dsa_87/signer.go`	

Description

The ML-DSA-87 implementation does not support the hedged signing variant. Both constructors hardcode `randomizedSigning` to false, no public setter exists, and the hedged code path is marked as dead code. The `crypto.Signer` wrapper additionally discards any caller-supplied `rand io.Reader` via a blank identifier, making it impossible to enable hedged signing. The lack of fresh randomness during signing adds the risk of side-channel attacks and fault-injection attacks to the implementation when compared with the hedged version.

```
    func NewMLDSA87FromSeed(seed [SEED_BYTES]uint8) (*MLDSA87, error) {
       var sk [CRYPTO_SECRET_KEY_BYTES]uint8
       var pk [CRYPTO_PUBLIC_KEY_BYTES]uint8

       if _, err := cryptoSignKeypair(&seed, &pk, &sk); err != nil {
              //coverage:ignore
              //rationale: cryptoSignKeypair only fails if sha3 operations fail,
which never happens
              return nil, err
       }

       return &MLDSA87{pk, sk, seed, false}, nil
    }
```

Figure 6.1: go-qrllib/crypto/ml_dsa_87/ml_dsa_87.go#L70–L81

```
    if randomizedSigning {
       //coverage:ignore
       //rationale: randomizedSigning is always false in current API (deterministic
signing)
       _, err := rand.Read(rnd[:])
       if err != nil {
              //coverage:ignore
              //rationale: crypto/rand.Read only fails if system entropy source is
broken
              return cryptoerrors.ErrSeedGeneration
       }
    }
```

Figure 6.2: go-qrllib/crypto/ml_dsa_87/sign.go#L283–L292

```
    func (s *CryptoSigner) Sign(_ io.Reader, digest []byte, opts crypto.SignerOpts)
([]byte, error) {
       var ctx []byte
```

Figure 6.3: go-qrllib/crypto/ml_dsa_87/signer.go#L56–L57

Recommendations

Short term, enable hedged signing for users. It mitigates any potential side-channel attacks and fault injection attacks with essentially no overhead and interoperability limitations.

7. SHAKE_128 is a non-compliant FIPS parameter with reduced quantum security

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Cryptography	Finding ID: TOB-QRLLIB-7
Target: `crypto/xmss/hashfunction.go`, `crypto/xmss/hash.go`, `legacywallet/xmss/wallet.go`	

Description

The XMSS implementation supports three hash functions: SHA2_256, SHAKE_128, and SHAKE_256. Notably, SHAKE_128 is not among the FIPS-approved parameter sets for XMSS, as NIST SP 800-208 exclusively defines SHA2 and SHAKE256. While SHAKE128 with a 32-byte output offers approximately 64-bit quantum security, this theoretical reduction remains difficult to exploit in practice. However, it may be worth flagging for users that there is reduced security when using SHAKE_128.

```
const (
	SHA2_256 HashFunction = iota
	SHAKE_128
	SHAKE_256
    )
```

Figure 7.1: go-qrllib/crypto/xmss/hashfunction.go#L11–L15

```
	case SHAKE_128:
		misc.SHAKE128(out, buf)
	case SHAKE_256:
		misc.SHAKE256(out, buf)
	case SHA2_256:
		misc.SHA256(out, buf)
	}
}
```

Figure 7.2: go-qrllib/crypto/xmss/hash.go#L46–L54

Recommendations

Short term, add documentation to SHAKE_128 in the HashFunction enum and in the legacywallet constructors clearly stating that it is a non-standard extension not defined in NIST SP 800-208, and that it provides reduced security compared to other options.

8. Exported XMSSWallet API drops security-critical documentation

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Cryptography	Finding ID: TOB-QRLLIB-8
Target: `legacywallet/xmss/wallet.go`, `crypto/xmss/xmss.go`, `crypto/xmss/doc.go`	

Description

The `XMSSWallet.Sign` function is a wallet wrapper that delegates to the inner `crypto/xmss.Sign`. The inner package carries a critical warning (figure 8.2) requiring callers to persist the updated OTS index to durable storage after signing and before using the returned signature. However, the `XMSSWallet.Sign` function (figure 8.1) does not carry the same warning, and a caller at the legacywallet API level can only discover the persistence requirement by navigating to `crypto/xmss/doc.go`.

This is also true for other limitations of the XMSS sign function, such as concurrent signing.

```
    func (w *XMSSWallet) Sign(message []uint8) ([]uint8, error) {
       return w.xmss.Sign(message)
    }
```

Figure 8.1: go-qrllib/legacywallet/xmss/wallet.go#L139–L141

```
// The caller MUST persist the updated index (via GetIndex) to durable storage
// before using the returned signature. See the package documentation for details.
func (x *XMSS) Sign(message []uint8) ([]uint8, error) {
       index := x.GetIndex()
       if err := x.SetIndex(index); err != nil {
              return nil, cryptoerrors.ErrSigningFailed
       }

       return xmssFastSignMessage(x.hashFunction, x.xmssParams, x.sk, x.bdsState,
message)
}
```

Figure 8.2: go-qrllib/crypto/xmss/xmss.go#L98–L108

```
//
// XMSS is a STATEFUL signature scheme. Each signature uses a unique index
// that MUST NEVER be reused. Reusing an index allows an attacker to forge
// signatures for ANY message.
//
// # Security Requirements
```

```
//
// To use XMSS safely, you MUST:
//
//  1. NEVER sign with the same index twice
//  2. Persist the UPDATED index to durable storage AFTER signing and BEFORE using
any signature
//  3. NEVER sign concurrently from the same XMSS instance
//  4. NEVER restore from backup without ensuring index continuity
//  5. Plan for key rotation before index exhaustion (2^height signatures)
//
// # Recommendation
```

Figure 8.3: go-qrllib/crypto/xmss/doc.go#L5–L20

Recommendations

Short term, add godoc to `XMSSWallet.Sign` matching the CRITICAL WARNING in `crypto/xmss/doc.go`, explicitly naming `GetIndex()` as the value the caller must persist before using the returned signature.

9. C-to-Go translation can silently change semantics due to operator precedence differences

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Cryptography	Finding ID: TOB-QRLLIB-9
Target: `crypto/ml_dsa_87/poly.go`, `crypto/dilithium/poly.go`	

Description

C and Go have different operator precedence rules, and expressions ported verbatim from C can evaluate differently in Go. The ML-DSA `polyChkNorm` function contains the expression `t = a.coeffs[i] - (t & 2 * a.coeffs[i])`, which evaluates differently in the two languages:

- In C, `*` binds more tightly than `&`, so the expression evaluates as `t & (2 * a)`.
- In Go, `*` and `&` share a precedence class, so the expression evaluates as `(t & 2) * a`.

The current code still computes the intended result only because the preceding line constrains `t` to the special value domain `{0, -1}`.

```
        t = a.coeffs[i] - (t & 2 * a.coeffs[i])
        violation |= (B - 1 - t) >> 31
}
```

Figure 9.1: go-qrllib/crypto/ml_dsa_87/poly.go#L100–L102

Recommendations

Short term, rewrite the expression with explicit parentheses so the intended grouping is unambiguous in Go and no longer depends on the domain values used, and the reader understands C-versus-Go precedence discrepancies.

10. ML-DSA secret-memory cleanup is inconsistent across functions

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Data Exposure	Finding ID: TOB-QRLLIB-10
Target: `crypto/ml_dsa_87/sign.go`, `crypto/ml_dsa_87/ml_dsa_87.go`	

Description

The `ML-DSA` implementation applies best-effort zeroization to secret state, but equivalent secret state remains unwiped in other sensitive paths. The `cryptoSignSignatureInternal` function (figure 10.1) defers cleanup of unpacked secret-key material such as `key`, `rhoPrime`, `s1`, `s2`, while `cryptoSignKeypair` derives and handles comparable secret intermediates such as `key`, `rhoPrime`, `s1`, `s1hat`, and `s2`, and returns without wiping them after packing.

```
var rho, key [SEED_BYTES]uint8
var tr [TR_BYTES]uint8
var mu, rhoPrime [CRH_BYTES]uint8
var s1, y, z polyVecL
var mat [K]polyVecL
var s2, t0, w1, h, w0 polyVecK
var cp poly
var nonce uint16

unpackSk(&rho, &tr, &key, &t0, &s1, &s2, sk)

// Zeroize secret temporaries when signing completes.
// Go's GC may copy values before zeroization, but this still reduces
// the window for secrets persisting in freed memory.
defer func() {
	zeroBytes(key[:])
	zeroBytes(rhoPrime[:])
	zeroPolyVecL(&s1)
	zeroPolyVecK(&s2)
	zeroPolyVecK(&t0)
}()
```

Figure 10.1: Deferred cleanup of secret state (go-qrllib/crypto/ml_dsa_87/sign.go#L133–L154)

```
	var tr [TR_BYTES]uint8
	var rho, key [SEED_BYTES]uint8
	var rhoPrime [CRH_BYTES]uint8

	var mat [K]polyVecL
```

```
	var s1, s1hat polyVecL
	var s2, t1, t0 polyVecK

	...

	/* Matrix-vector multiplication */
	s1hat = s1
	polyVecLNTT(&s1hat)
	polyVecMatrixPointWiseMontgomery(&t1, &mat, &s1hat)
	polyVecKReduce(&t1)
	polyVecKInvNTTToMont(&t1)

	/* Add noise vector s2 */
	polyVecKAdd(&t1, &t1, &s2)

	/* Extract t1 and write public key */
	polyVecKCAddQ(&t1)
	polyVecKPower2Round(&t1, &t0, &t1)
	packPk(pk, rho, &t1)

	/* Compute tr = CRH(rho, t1) and write secret key */
	sha3.ShakeSum256(tr[:], pk[:])
	packSk(sk, rho, tr, key, &t0, &s1, &s2)

	return seed, nil
}
```

Figure 10.2: Key generation function does not clean local state after packing the secret key. (go-qrllib/crypto/ml_dsa_87/sign.go#L42–L130)

The `NewMLDSA87FromHexSeed` function also decodes a caller-supplied secret seed into a temporary byte slice and leaves that copy resident.

```
	if strings.HasPrefix(hexSeed, "0x") || strings.HasPrefix(hexSeed, "0X") {
		hexSeed = hexSeed[2:]
	}
	unsizedSeed, err := hex.DecodeString(hexSeed)
	if err != nil {
		return nil, fmt.Errorf(common.ErrDecodeHexSeed, wallettype.ML_DSA_87,
err.Error())
	}
	if len(unsizedSeed) != SEED_BYTES {
		return nil, cryptoerrors.ErrInvalidSeed
	}
	var seed [SEED_BYTES]uint8
	copy(seed[:], unsizedSeed)
	return NewMLDSA87FromSeed(seed)
}
```

Figure 10.3: go-qrllib/crypto/ml_dsa_87/ml_dsa_87.go#L84–L97

The public `Zeroize` method overwrites fields directly instead of reusing the package's `zeroBytes` helper:

```
// This should be called when the MLDSA87 instance is no longer needed.
func (d *MLDSA87) Zeroize() {
	for i := range d.sk {
		d.sk[i] = 0
	}
	for i := range d.seed {
		d.seed[i] = 0
	}
}
```

Figure 10.4: go-qrllib/crypto/ml_dsa_87/ml_dsa_87.go#L170–L179

```
// from eliding the writes as a dead store.
func zeroBytes(b []byte) {
	for i := range b {
		b[i] = 0
	}
	runtime.KeepAlive(&b)
}
```

Figure 10.5: go-qrllib/crypto/ml_dsa_87/sign.go#L13–L26

Recommendations

Short term, apply the same best-effort zeroization pattern consistently across sensitive paths, including deferred cleanup for secret intermediates in `cryptoSignKeypair`, wiping temporary decoded seed buffers in `NewMLDSA87FromHexSeed`, and making `MLDSA87.Zeroize()` reuse the same helper functions used elsewhere in the package.

Long term, document the exact guarantee boundary of zeroization in Go. Avoid returning raw secret keys and seed copies unless required, centralize wiping helpers in a single place, and describe zeroization as a best-effort measure rather than a guarantee of complete erasure.

11. ML-DSA Open and Verify panic on a nil public key

Severity: **Low**	Difficulty: **Low**
Type: Denial of Service	Finding ID: TOB-QRLLIB-11
Target: `crypto/ml_dsa_87/ml_dsa_87.go`, `crypto/ml_dsa_87/sign.go`, `crypto/ml_dsa_87/packing.go`	

Description

The public ML-DSA `Open` and `Verify` functions do not validate that the supplied public key pointer is non-nil before attempting verification. Both functions pass the public key into the shared verification and public-key unpacking logic, and the code dereferences that pointer, which triggers a runtime panic. The same issue extends to the `Verify` function in `wallet/ml_dsa_87`.

```
func unpackPk(rho *[SEED_BYTES]uint8,
	t1 *polyVecK,
	pkb *[CRYPTO_PUBLIC_KEY_BYTES]uint8) {
	pk := pkb[:]
	copy(rho[:], pk[:])
	pk = pk[SEED_BYTES:]
	for i := 0; i < K; i++ {
		polyT1Unpack(&t1.vec[i], pk[i*POLY_T1_PACKED_BYTES:])
	}
}
```

Figure 11.1: go-qrllib/crypto/ml_dsa_87/packing.go#L14–L23

```
// In case the signature is invalid, nil is returned.
func Open(ctx, signatureMessage []uint8, pk *[CRYPTO_PUBLIC_KEY_BYTES]uint8) []uint8
{
	msg, _ := cryptoSignOpen(signatureMessage, ctx, pk)
	return msg
}

// Verify checks the signature against the message and public key with the given
context.
// The ctx parameter must match the context used during signing (FIPS 204
requirement).
func Verify(ctx, message []uint8, signature [CRYPTO_BYTES]uint8, pk
*[CRYPTO_PUBLIC_KEY_BYTES]uint8) bool {
	result, err := cryptoSignVerify(signature, message, ctx, pk)
	if err != nil {
		return false
	}
```

```
	return result
}
```

Figure 11.2: go-qrllib/crypto/ml_dsa_87/ml_dsa_87.go#L135–L149

```
func cryptoSignOpen(sm []uint8, ctx []uint8, pk *[CRYPTO_PUBLIC_KEY_BYTES]uint8)
([]uint8, error) {
	if len(sm) < CRYPTO_BYTES {
		return nil, nil
	}

	var sig [CRYPTO_BYTES]uint8
	msg := make([]uint8, len(sm)-CRYPTO_BYTES)

	copy(sig[:], sm)
	copy(msg, sm[CRYPTO_BYTES:])

	if result, err := cryptoSignVerify(sig, msg, ctx, pk); err != nil || !result {
		return nil, err
	}

	return msg, nil
}
```

Figure 11.3: go-qrllib/crypto/ml_dsa_87/sign.go#L410–L438

The following test confirms the issue:

```
func TestEdgeCaseNilPublicKeyPanics(t *testing.T) {
	mldsa, err := New()
	if err != nil {
		t.Fatalf("Failed to create MLDSA87: %v", err)
	}

	ctx := []byte("test-context")
	msg := []byte("test message")

	sig, err := mldsa.Sign(ctx, msg)
	if err != nil {
		t.Fatalf("Failed to sign: %v", err)
	}

	sealed, err := mldsa.Seal(ctx, msg)
	if err != nil {
		t.Fatalf("Failed to seal: %v", err)
	}

	t.Run("Verify", func(t *testing.T) {
		defer func() {
			if recover() == nil {
				t.Fatal("Verify should panic on a nil public key")
			}
```

```
                }()

                _ = Verify(ctx, msg, sig, nil)
        })

        t.Run("Open", func(t *testing.T) {
                defer func() {
                        if recover() == nil {
                                t.Fatal("Open should panic on a nil public key")
                        }
                }()

                _ = Open(ctx, sealed, nil)
        })
}
```

Figure 11.4: Test case for panicking `Verify` and `Open` functions on nil public keys

Exploit Scenario

An application exposes `ML-DSA` signature verification through an interface where the public key may be a `nil` pointer. The application forwards that `nil` public key into `Open` or `Verify`, and the process panics with a nil-pointer dereference instead of rejecting the input, interrupting service availability.

Recommendations

Short term, add an explicit `nil` check in `Open`, `Verify`, `cryptoSignOpen`, `cryptoSignVerify`, and to the `Verify` function in `wallet/ml_dsa_87`, and return failure without panicking when the public key is `nil`.

Long term, add regression tests that verify the correct handling of malformed inputs to public APIs.

12. ML-DSA Seal naming implies confidentiality for a signature-only operation

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Error Reporting	Finding ID: TOB-QRLLIB-12
Target: `crypto/ml_dsa_87/ml_dsa_87.go`, `crypto/ml_dsa_87/example_test.go`	

Description

The public `ML-DSA` API uses the name `Seal` for an operation that returns the signature and the signed message in plaintext, `signature || message`. `Seal` commonly refers to authenticated encryption interfaces such as AEADs. Using that term for a signature-only operation can mislead API users into assuming confidentiality properties that the function does not provide.

```
// Seal the message, returns signature attached with message.
func (d *MLDSA87) Seal(ctx, message []uint8) ([]uint8, error) {
	return cryptoSign(message, ctx, &d.sk, d.randomizedSigning)
}
```

Figure 12.1:go-qrllib/crypto/ml_dsa_87/ml_dsa_87.go#L116–L119

The `Seal` usage example also hints at the sealed data being confidential by using `"confidential data"` as the signed message.

```
func ExampleMLDSA87_Seal() {
	m, _ := ml_dsa_87.New()
	defer m.Zeroize()

	ctx := []byte("seal-context")
	message := []byte("confidential data")

	// Seal prepends the signature to the message
	sealed, err := m.Seal(ctx, message)
	if err != nil {
```

Figure 12.2: go-qrllib/crypto/ml_dsa_87/example_test.go#L113–L122

Recommendations

Short term, consider renaming the API to a term that reflects its actual behavior, such as `SignAttached` or `AttachSignature`, and update examples to avoid message names that imply confidentiality.

13. Invalid XMSS hash function values produce degenerate interchangeable keys and signatures

Severity: **High**	Difficulty: **High**
Type: Cryptography	Finding ID: TOB-QRLLIB-13
Target: `crypto/xmss/xmss.go`, `crypto/xmss/hash.go`, `legacywallet/xmss/wallet.go`	

Description

The XMSS public API accepts invalid `HashFunction` enum values and continues key generation with a degenerate hashing primitive. This behavior breaks the XMSS signature API: unsupported hash function values cause zero-root generation, which in turn makes distinct keys produce identical signatures that cross-verify under each other's public keys.

The `InitializeTree` function forwards the `hashFunction` to `XMSSFastGenKeyPair` without validating that it is one of the supported values. Then, `coreHash` dispatches on that enum with a `switch` that has no `default` case. When callers pass an unsupported value, none of the cases run and the output buffers remain zero-initialized.

```
func InitializeTree(h Height, hashFunction HashFunction, seed []uint8) (*XMSS,
error) {
	height := uint32(h)
	sk := make([]uint8, 132)
	pk := make([]uint8, 64)

	k := WOTSParamK
	w := WOTSParamW
	n := WOTSParamN

	if k >= height || (height-k)%2 == 1 {
		return nil, cryptoerrors.ErrInvalidBDSParams
	}

	xmssParams := NewXMSSParams(n, height, w, k)
	bdsState := NewBDSState(height, n, k)

	if err := XMSSFastGenKeyPair(hashFunction, xmssParams, pk, sk, bdsState,
seed); err != nil {
```

Figure 13.1:go-qrllib/crypto/xmss/xmss.go#L24–L40

```
func coreHash(hashFunction HashFunction, out []uint8, typeValue uint32, key []uint8,
keyLen uint32, in []uint8, inLen uint32, n uint32) {
```

```
	buf := make([]uint8, inLen+n+keyLen)
	misc.ToByteBigEndian(buf, typeValue, n) // RFC 8391 requires big-endian
encoding

	for i := uint32(0); i < keyLen; i++ {
		buf[i+n] = key[i]
	}

	for i := uint32(0); i < inLen; i++ {
		buf[keyLen+n+i] = in[i]
	}

	switch hashFunction {
	case SHAKE_128:
		misc.SHAKE128(out, buf)
	case SHAKE_256:
		misc.SHAKE256(out, buf)
	case SHA2_256:
		misc.SHA256(out, buf)
	}
}
```

Figure 13.2: go-qrllib/crypto/xmss/hash.go#L34–L53

The same issue is reachable through the `NewWalletFromSeed` API, as it performs no validation of the hash function.

```
func NewWalletFromSeed(seed [SeedSize]uint8, height xmss.Height, hashFunction
xmss.HashFunction, addrFormatType common.AddrFormatType) (*XMSSWallet, error) {
	signatureType := legacywallet.WalletTypeXMSS // Signature Type hard coded for
now
	if height > xmss.MaxHeight {
		return nil, fmt.Errorf("height %d exceeds maximum %d", height,
xmss.MaxHeight)
	}
	desc := NewQRLDescriptor(height, hashFunction, signatureType, addrFormatType)

	tree, err := xmss.InitializeTree(desc.height, desc.hashFunction, seed[:])
	if err != nil {
		return nil, fmt.Errorf("failed to initialize XMSS tree: %w", err)
	}

	return &XMSSWallet{
		seed: seed,
		desc: desc,
		xmss: tree,
	}, nil
}
```

Figure 13.3: go-qrllib/legacywallet/xmss/wallet.go#L21–L38

Exploit Scenario

An application exposes XMSS parameter selection through configuration. An attacker supplies an unsupported hash function value. Instead of rejecting the request, the library generates an XMSS instance with a constant zero root. Distinct seeds then yield interchangeable public keys and signatures under the invalid enum value, and the application uses key material that no longer satisfies XMSS security guarantees.

Recommendations

Short term, reject unsupported hash function values at every public XMSS constructor, including `InitializeTree` and `NewWalletFromSeed`, by checking `hashFunction.IsValid()` and returning an error on failure.

Long term, add regression tests that verify invalid enums are rejected for all public APIs rather than silently producing degenerate roots and interchangeable signatures.

14. ML-DSA Open API collapses distinct failure modes into a nil result

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Error Reporting	Finding ID: TOB-QRLLIB-14
Target: `crypto/ml_dsa_87/ml_dsa_87.go`, `crypto/ml_dsa_87/sign.go`	

Description

The public `Open` API returns only a message slice and uses `nil` to represent every failure mode. The wrapper discards the error returned by `cryptoSignOpen` and returns only the recovered message. As a result, malformed attached-signature inputs, oversized contexts, verification failure, and internal verification errors are all collapsed into the same `nil` result.

```
// In case the signature is invalid, nil is returned.
func Open(ctx, signatureMessage []uint8, pk *[CRYPTO_PUBLIC_KEY_BYTES]uint8) []uint8
{
	msg, _ := cryptoSignOpen(signatureMessage, ctx, pk)
	return msg
}
```

Figure 14.1: go-qrllib/crypto/ml_dsa_87/ml_dsa_87.go#L135–L139

Recommendations

Short term, consider redesigning the `Open` function to return `([]byte, error)` so the API follows standard Go error-reporting patterns and preserves verification-path failure context for callers.

Long term, add unit tests that exercise the error-return cases.

15. Unpinned external GitHub CI/CD action versions

Severity: **Low**	Difficulty: **High**
Type: Auditing and Logging	Finding ID: TOB-QRLLIB-15
Target: `.github/workflows/actionlint.yml`, `.github/workflows/lint.yml`, `.github/workflows/release.yml`, `.github/workflows/test.yml`	

Description

The GitHub Actions in the `go-qrllib` repository use third-party actions pinned to a tag instead of a full commit SHA as recommended by GitHub. This configuration enables repository owners to silently modify the actions. A malicious actor could use this ability to leak secrets.

The following actions are owned by organizations or individuals that are not affiliated directly with QRL or GitHub:

- `raven-actions/actionlint@v2`
- `golangci/golangci-lint-action@v9`
- `go-semantic-release/action@v1`
- `anchore/sbom-action@v0`
- `codecov/codecov-action@v5`

Exploit Scenario

An attacker gains unauthorized access to the account of a GitHub action owner. The attacker manipulates the action's code to steal GitHub credentials and compromise the repository.

Recommendations

Short term, pin each third-party action to a specific full-length commit SHA, as GitHub recommends. Additionally, configure Dependabot to update the actions' commit SHAs after reviewing their available updates.

Long term, add `zizmor` to the CI/CD pipeline.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Issues

This appendix contains findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Duplicate implementations under different APIs.** The `NewQRLDescriptorFromExtendedPK` and `LegacyQRLDescriptorFromExtendedPK` functions call `NewQRLDescriptorFromBytes` and `LegacyQRLDescriptorFromBytes`, respectively. However, `NewQRLDescriptorFromBytes` and `LegacyQRLDescriptorFromBytes` are duplicates of each other. We suggest either removing one, or implementing one as a thin wrapper of the other, instead of keeping the duplicate implementations.

```
func NewQRLDescriptorFromBytes(descriptorBytes []uint8) (*QRLDescriptor,
error) {
	if len(descriptorBytes) != 3 {
		return nil, fmt.Errorf("%w: got %d", ErrInvalidDescriptorSize,
len(descriptorBytes))
	}

	hashFunction, err := xmss.ToHashFunction(descriptorBytes[0] & 0x0f)
	if err != nil {
		return nil, fmt.Errorf("invalid hash function in descriptor:
%w", err)
	}

	height, err := xmss.ToHeight((descriptorBytes[1] & 0x0f) << 1)
	if err != nil {
		return nil, fmt.Errorf("invalid height in descriptor: %w", err)
	}

	signatureType, err := legacywallet.ToWalletType((descriptorBytes[0] >>
4) & 0x0F)
	if err != nil {
		return nil, fmt.Errorf("invalid signature type in descriptor:
%w", err)
	}

	return &QRLDescriptor{
		hashFunction:   hashFunction,
		signatureType:  signatureType,
		height:         height,
		addrFormatType: common.AddrFormatType((descriptorBytes[1] &
0xF0) >> 4),
	}, nil
}
```

```
func LegacyQRLDescriptorFromBytes(descriptorBytes []uint8) (*QRLDescriptor,
error) {
	if len(descriptorBytes) != 3 {
		return nil, fmt.Errorf("%w: got %d", ErrInvalidDescriptorSize,
len(descriptorBytes))
	}

	hashFunction, err := xmss.ToHashFunction(descriptorBytes[0] & 0x0f)
	if err != nil {
		return nil, fmt.Errorf("invalid hash function in descriptor:
%w", err)
	}

	height, err := xmss.ToHeight((descriptorBytes[1] & 0x0f) << 1)
	if err != nil {
		return nil, fmt.Errorf("invalid height in descriptor: %w", err)
	}

	signatureType, err := legacywallet.ToWalletType((descriptorBytes[0] >>
4) & 0x0F)
	if err != nil {
		return nil, fmt.Errorf("invalid signature type in descriptor:
%w", err)
	}

	return &QRLDescriptor{
		hashFunction:   hashFunction,
		signatureType:  signatureType,
		height:         height,
		addrFormatType: common.AddrFormatType((descriptorBytes[1] &
0xF0) >> 4),
	}, nil
}
```

Figure B.1: go-qrllib/legacywallet/xmss/descriptor.go#L43–L97

C. Automated Analysis Tool Configuration

This appendix describes the setup of the automated analysis tools used during this audit.

Though static analysis tools frequently report false positives, they detect certain categories of issues, such as memory leaks, misspecified format strings, and the use of unsafe APIs, with essentially perfect precision. We recommend periodically running these static analysis tools and reviewing their findings.

We convert, when possible, tool output to the SARIF file format (e.g., with `clippy-sarif`), allowing for easy inspection of the results within an IDE (e.g., using VS Code's SARIF Explorer extension).

CodeQL

We used CodeQL to detect known vulnerabilities in the codebase. Install CodeQL by following CodeQL's installation guide.

```
# Create the Go database
codeql database create codeql.db --language=go

# Run all queries for a language
codeql database analyze codeql.db --format=sarif-latest --output=codeql.sarif
```

Figure C.1: The commands used to run CodeQL on the codebase

During the engagement, we ran the following query packs and query suites on the codebase:

- The `trailofbits/go-queries` query pack
- The `codeql/go-queries` query pack (included with CodeQL)
- The Go `security-extended` query suite (included with CodeQL)

We also ran internally maintained CodeQL query suites on the codebase. For more information about CodeQL, refer to the CodeQL chapter of the Trail of Bits Testing Handbook.

Semgrep

We ran the static analyzer Semgrep with the rulesets shown in figure C.2 to try and identify low-complexity weaknesses in the codebase. To install Semgrep, we used `pip` by running `python3 -m pip install semgrep`.

```
git clone git@github.com:dgryski/semgrep-go.git
```

```
semgrep --metrics=off --sarif --config p/ci
semgrep --metrics=off --sarif --config p/golang
semgrep --metrics=off --sarif --config p/trailofbits
semgrep --metrics=off --sarif --config p/security-audit
semgrep --metrics=off --sarif --config semgrep-go
```

Figure C.2: The command and rulesets used to run Semgrep on the codebase

zizmor

Zizmor is a static analysis tool for GitHub Actions. It can be installed using `cargo install --locked zizmor`. We ran zizmor with the following command:

```
zizmor --persona pedantic . --format sarif > zizmor_pedantic.sarif
```

Figure C.3: Command used to run zizmor and output to SARIF

golangci-lint

The tool `golangci-lint` allows users to efficiently run a set of linters of Go code. To run `golangci-lint`, use the following command: `golangci-lint run ./...`

govulncheck

We used `govulncheck` to identify vulnerable dependencies in the Go codebase using the following command: `govulncheck ./...`.

Go cover

We ran the go cover tool on both the `trie` and `types` folders to obtain a test coverage report with the following command:

```
go test -cover -coverprofile c.out && go tool cover -html=c.out
```

Figure C.4: Command used to obtain test coverage and visualize it in the browser

Note that this needs to be run separately for each folder.

Gremlins

We ran the Gremlins tool using the following command:

```
gremlins unleash --output reports/gremlins/gremlins.json .
```

Figure C.5: Command used to run mutation tests on the codebase

D. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On June 22, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the The Quantum Resistant Ledger team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, of the 15 issues described in this report, The Quantum Resistant Ledger team has resolved all 15 issues. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	XMSS implementation is not fully compliant with RFC 8391	Low	Resolved
2	XMSS InitializeTree accepts invalid typed Height values above MaxHeight	Low	Resolved
3	Modern wallet signatures do not bind descriptor metadata	Informational	Resolved
4	Wallet allows SPHINCS+ as valid signature scheme	Informational	Resolved
5	prf function hardcodes input length without validating the actual input length	Informational	Resolved
6	ML-DSA implementation uses deterministic signing	Informational	Resolved
7	SHAKE_128 is a non-compliant FIPS parameter with reduced quantum security	Informational	Resolved
8	Exported XMSSWallet API drops security-critical documentation	Informational	Resolved

9	C-to-Go translation can silently change semantics due to operator precedence differences	Informational	Resolved
10	ML-DSA secret-memory cleanup is inconsistent across functions	Informational	Resolved
11	ML-DSA Open and Verify panic on a nil public key	Low	Resolved
12	ML-DSA Seal naming implies confidentiality for a signature-only operation	Informational	Resolved
13	Invalid XMSS hash function values produce degenerate interchangeable keys and signatures	High	Resolved
14	ML-DSA Open API collapses distinct failure modes into a nil result	Informational	Resolved
15	Unpinned external GitHub CI/CD action versions	Low	Resolved

Detailed Fix Review Results

TOB-QRLLIB-1: XMSS implementation is not fully compliant with RFC 8391
Resolved in e1bcea3, 526acc4. Unsupported parameter tuples are now rejected.

TOB-QRLLIB-2: XMSS InitializeTree accepts invalid typed Height values above MaxHeight
Resolved in 1553251. The height is now validated, and regression tests were added.

TOB-QRLLIB-3: Modern wallet signatures do not bind descriptor metadata
Resolved in dbb5168, 7497482. Descriptor metadata is strictly validated and bound to the signing context.

TOB-QRLLIB-4: Wallet allows SPHINCS+ as valid signature scheme
Resolved in 8d51787, 7fada1c. SPHINCS+ was made invalid in wallet validation.

TOB-QRLLIB-5: prf function hardcodes input length without validating the actual input length
Resolved in 3f8aec9. The input type was changed, preventing the issue at compile time.

TOB-QRLLIB-6: ML-DSA implementation uses deterministic signing
Resolved in 6c61398. The hedged version is now used by default, still allowing the deterministic version via a separate API for specific uses (e.g., testing).

TOB-QRLLIB-7: SHAKE_128 is a non-compliant FIPS parameter with reduced quantum security
Resolved in f14db2f. Documentation was added clearly indicating that SHAKE_128 is not recommended for new wallets, and is present only for legacy compatibility.

TOB-QRLLIB-8: Exported XMSSWallet API drops security-critical documentation
Resolved in 27f5e26. Documentation was added to the wrapper API, preserving the security-critical documentation.

TOB-QRLLIB-9: C-to-Go translation can silently change semantics due to operator precedence differences
Resolved in 4ecdaea. Documentation and parentheses were added to ensure that semantics are preserved.

TOB-QRLLIB-10: ML-DSA secret-memory cleanup is inconsistent across functions
Resolved in 14b5188. Secret memory cleanup was made consistent across functions by adding deferred cleanup to buffers containing secret data.

TOB-QRLLIB-11: ML-DSA Open and Verify panic on a nil public key
Resolved in 36788e3. Documentation and public key validation was added to several functions, preventing runtime errors in case a nil public key was provided.

TOB-QRLLIB-12: ML-DSA Seal naming implies confidentiality for a signature-only operation
Resolved in 49238aa, 7fada1c. The function was renamed and documentation was added clearly stating that the API does not provide confidentiality.

TOB-QRLLIB-13: Invalid XMSS hash function values produce degenerate interchangeable keys and signatures
Resolved in fa42a77, 7fada1c. Validation was added to the selected hash function, and a default case in the hash function selection was added.

TOB-QRLLIB-14: ML-DSA Open API collapses distinct failure modes into a nil result
Resolved in 8568b3a. The error value is now returned in the Open API.

TOB-QRLLIB-15: Unpinned external GitHub CI/CD action versions
Resolved in a32df75. External GitHub action versions are now pinned, and Zizmor was added to CI.

E. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to The Quantum Resistant Ledger under the terms of the project statement of work and has been made public at The Quantum Resistant Ledger's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the Trail of Bits Publications page. Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.